

## models\cve\_model.py

```
# Type hints for better code documentation and IDE support
from typing import Dict, List, Any, Optional
# Date/time handling for parsing CVE publication dates
from datetime import datetime

class CVEModel:
    """Data model for CVE vulnerability records"""

    def __init__(self, data: Dict[str, Any]):
        # Extract CVE identifier (e.g., "CVE-2023-12345") with empty string fallback
        self.id = data.get('ID', '')
        # Get vulnerability description/summary with empty string fallback
        self.description = data.get('Description', '')
        # Extract severity level with 'UNKNOWN' fallback for proper classification
        self.severity = data.get('Severity', 'UNKNOWN')
        # Get CVSS numeric score (can be None if not assessed)
        self.cvss_score = data.get('CVSS_Score')
        # Extract CWE weakness identifier (can be None if not mapped)
        self.cwe = data.get('CWE')
        # Get publication date string with empty string fallback
        self.published = data.get('Published', '')
        # Extract last modified date string with empty string fallback
        self.last_modified = data.get('lastModified', '')
        # Get reference URLs list with empty list fallback
        self.references = data.get('References', [])
        # Extract affected products list with empty list fallback
        self.products = data.get('Products', [])
        # Get CVSS metrics dictionary with empty dict fallback
        self.metrics = data.get('metrics', {})

    def to_dict(self) -> Dict[str, Any]:
        """Convert to dictionary format"""
        # Return all instance attributes as dictionary for JSON serialization
        return {
            'ID': self.id,
            'Description': self.description,
            'Severity': self.severity,
            'CVSS_Score': self.cvss_score,
            'CWE': self.cwe,
            'Published': self.published,
            'lastModified': self.last_modified,
            'References': self.references,
            'Products': self.products,
            'metrics': self.metrics
        }

    def get_published_date(self) -> Optional[datetime]:
        """Get published date as datetime object"""
        # Check if published date string exists
        if self.published:
            try:
                # Handle ISO format with time (e.g., "2023-01-15T10:30:00Z")
                if 'T' in self.published:
                    # Replace Z with timezone offset for proper parsing
                    return datetime.fromisoformat(self.published.replace('Z', '+0000'))
                else:
                    # Handle date-only format (e.g., "2023-01-15")
                    return datetime.strptime(self.published[:10], '%Y-%m-%d')
            except:
```

```
# Return None if any parsing error occurs  
return None  
return None
```

```
def is_critical(self) -> bool:  
    """Check if CVE is critical severity"""  
    # Case-insensitive check for exactly 'CRITICAL' severity  
    return self.severity.upper() == 'CRITICAL'
```

```
def is_high_priority(self) -> bool:  
    """Check if CVE is critical or high severity"""  
    # Case-insensitive check for either 'CRITICAL' or 'HIGH' severity  
    return self.severity.upper() in ['CRITICAL', 'HIGH']
```